

本小节内容

指针的定义

取地址操作符与取值操作符，指针本质

1 指针的定义

内存区域中的每字节都对应一个编号，这个编号就是“地址”。如果在程序中定义了一个变量，那么在对程序进行编译时，系统就会给这个变量分配内存单元。按变量地址存取变量值的方式称为“直接访问”，如 `printf("%d",i);`、`scanf("%d",&i);`等；另一种存取变量值的方式称为“间接访问”，即将变量 `i` 的地址存放到另一个变量中。在 C 语言中，指针变量是一种特殊的变量，它用来存放变量地址。

指针变量的定义格式如下：

基类型 *指针变量名；

例如，

```
int *i_pointer;
```

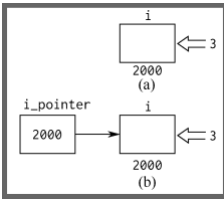


图 指针变量

指针与指针变量是两个概念，一个变量的地址称为该变量的“指针”。例如，地址 2000 是变量 `i` 的指针。如果有一个变量专门用来存放另一变量的地址（即指针），那么称它为“指针变量”。例如，左图中的 `i_pointer` 就是一个指针变量。

那么 `i_pointer` 本身占多大的内存空间呢？本章中编写的程序都是 64 位应用程序，寻址范围为 64 位即 8 字节，所以对于本章来说 `sizeof(i_pointer)=8`。如果编写的程序是 32 位，那么寻址范围就是 4 字节（考研中往往会强调程序是 32 位的程序）。

2 取地址操作符与取值操作符，指针本质

取地址操作符为 `&`，也称引用，通过该操作符我们可以获取一个变量的地址值；取值操作符为 `*`，也称解引用，通过该操作符我们可以得到一个地址对应的数据。如下例所示，我们通过 `&i` 获取整型变量 `i` 的地址值，然后对整型指针变量 `p` 进行初始化，`p` 中存储的是整型变量 `i` 的地址值，所以通过 `*p` (`printf` 函数中的 `*p`) 就可以获取整型变量 `i` 的值。`p` 中存储的是一个绝对地址值，那为什么取值时会获取 4 字节大小的空间呢？这是因为 `p` 为整型变量指针，每个 `int` 型数据占用 4 字节大小的空间，所以 `p` 在解引用时会访问 4 字节大小的空间，同时以整型值对内存进行解析。

【例】 取地址与取值（引用与解引用）

```
#include <stdio.h>
```

//&符号是取地址，指针变量的初始化一定是某个变量取地址

```
int main()
{
    int i = 5;
    int* p=&i;
    printf("i=%d\n", i); //直接访问
    printf("*p=%d\n", *p); //间接访问
    return 0;
}
```

读者需要注意以下 4 点。

(1) 指针变量前面的 “*” 表示该变量为指针型变量。例如，

```
float *pointer_1;
```

注意指针变量名是 `pointer_1`，而不是 `*pointer_1`。

(2) 在定义指针变量时必须指定其类型。需要注意的是，只有整型变量的地址才能放到指向整型变量的指针变量中。例如，下面的赋值是错误的：

```
float a;
int * pointer_1;
pointer_1=&a; //毫无意义而且会出错，有兴趣的读者可以自行尝试
```

(3) 如果已执行了语句

```
pointer_1=&a;
```

那么 `&* pointer_1` 的含义是什么呢？

“&” 和 “*” 两个运算符的优先级相同，但要按自右向左的方向结合。因此，`&* pointer_1` 与 `&a` 相同，都表示变量 `a` 的地址，也就是 `pointer_1`。

`*&a` 的含义是什么呢？

首先进行 `&a` 运算，得到 `a` 的地址，再进行 `*` 运算。`*&a` 和 `*pointer_1` 的作用是一样的，它们都等价于变量 `a`，即 `*&a` 与 `a` 等价。

(4) C 语言本质上是一种自由形式的语言，这很容易诱使我们把 “*” 写在靠近类型的一侧，如 `int *a` 这个声明与前面一个声明具有相同的意思，而且看上去更清晰，`a` 被声明成类型为 `int*` 的指针。但是，这并不是一个好习惯，因为类似 `int *a,b,c` 的语句会使人们很自然地认为这条语句把所有三个变量声明为指向整型的指针，但事实上并非如此，“*” 实际上是 `*a` 的一部分，只对 `a` 标识符起作用，但其余两个变量只是普通的整型变量。要声明三个指针变量，正确的语句如下：

```
int *a,*b,*c;
```